

Introduction

This project is a fullstack web application which is designed to store, manage and visualise study session data which is generated by an AI based focus tracking system. The main focus of this application is to give users a clear way to review their study performance, in particular their focus levels over time during the session which is then turned into data produced by the study tracker.

The idea for my project came from the fact that the AI tracker I made only outputs raw JSON data, which is not at all user friendly, especially for users who don't know their way around a computer. To solve this issue I created this web application dashboard where this data can be imported, stored in a database and then presented in a much more meaningful and visual way. The idea I had was simple, the tracker spits out a JSON file, but that means nothing to most people. I wanted somewhere to actually make sense of all the data.

The system is built using a complete full stack approach which comes with a frontend interface to allow users to import and view session data and also a backend server that handles all of the data storage and retrieval. The backend is mostly responsible for managing a database of sessions and then exposing API endpoints while the frontend communicates with these endpoints in a way that can display the data so the user can make sense of it.

Design and development

Architecture design

When I started to build the fullstack side of this project I knew that I didn't want everything to be mixed together in one place because it would quickly become messy and difficult to manage. Instead I decided to split the project into a frontend and backend so that each part had its own responsibility.

The front end side handles everything that the user sees and interacts with such as viewing sessions, importing data and seeing the different statistics, however the backend is responsible for storing and retrieving session data using an API and a database. By separating these it made it much easier to debug issues and also meant I could work on each part independently without breaking any other part of the project.

```
client/  
  src/  
    components/  
    main.js  
    main-import.js  
    apiRouter.js  
    vite.config.js  
  
server/  
  index.js  
  
sessions.db
```

The overall flow of the system is very simple. The AI tracker generates session data as a JSON file at the end of a study session once the user has finished. That data is then copied

into the website using the import page. From there, the frontend processes the JSON and sends it to the backend through a POST request. The backend then stores the session in a SQLite database. When the sessions page is loaded, the frontend sends a GET request to retrieve all stored sessions and displays them along with calculated statistics and charts used with [chart.js](#).

The reason i went with this structure is because it reflects how real web applications handle data, even though the project is small and pretty simple. It also made it easier to extend later, for example when adding statistics and charts without needing to change how data is stored.

Frontend development

For the frontend, I structured the project into multiple pages rather than keeping everything on a single screen page. The main pages in this website are the sessions page, the import page, and the guide page. I chose to do this because it keeps each part of the application focused on one task instead of overwhelming the user with too much information at once i also think this layout is more professional rather than having everything cramped into one place.

Within each page, I broke the User Interface down into smaller components such as session tiles, the statistics display, and the chart. This made the code easier to manage and reuse. For example, each session is displayed using the same structure, so having a reusable component avoided repeating code and made updates much more simpler.

```
# sessionStats.css
JS sessionStats.js
# sessionTile.css
JS sessionTile.js
```

The session page is the main part of the application as it loads all of the session data from the backend and then displays it in a list along with different calculated statistics such as average focus score, total study time, best session, and number of sessions. I also added a simple chart using Chart.js to show how focus scores change over time. This wasn't very difficult or complex, but it makes the data easier to understand visually instead of just reading numbers.

```
Average Focus: 67.6%
Best Score: 85.5%
Total Study Time: 0h 27m
Sessions: 9
```

I used Vite as the development tool mainly because it made running the frontend locally really quick and easy. It also handled things like imports without needing extra any setup which helped keep the project pretty straightforward. So overall, the frontend was designed to be simple and clear to use, rather than overcomplicated. The focus was on making sure the user could easily import data, understand their results and understand how to setup the study tracker.

Backend Development

The backend for this project was built using [Express.js](#) and basically acts as a simple REST API which is used for handling the session data. I kept this pretty minimal with only really two main endpoints as that was all I needed for the application to function properly as intended.

The first endpoint is a POST request which is used in my project to add a new session data to the database. When the user imports their JSON data on the front end this then gets sent to the POST endpoint where it gets processed and stored. The second endpoint is a GET request which returns all of the stored sessions so that they can be displayed on the sessions page and in the charts in a way that the user can make sense of.

For the database side of things I used SQLite because I knew it was lightweight and doesn't really have any complicated setup. This made it very ideal for my projects like this where I didn't need a full database server. All of the session data is stored locally in a simple table structure which includes fields such as date, duration, focus score and username so that users can choose to add an optional name to their session.

```
app.post("/addSession", (req, res) => {  
  const { date, duration, focused, score, userName } = req.body;
```

I also setup CORS on the backend so that the frontend could communicate with it properly during my development. If I didn't have this setup then the browser would just block any requests between different ports.

I made the backend very simple even though it still handles all of the main functions that are needed to store and retrieve the user's session data reliably. I also made sure to add comments throughout the backend code, especially around the API endpoints and database logic, to make it clear what each part is doing.

Data integration

One of the main challenges in this project was definitely connecting the AI tracker data to the fullstack app. At the beginning the JSON files that were generated by the AI tracker didn't really match the exact format that the database I created expected, because of this I had to create a whole step to transform the data before sending it to the backend

```
const session = {  
  date: data.date,  
  duration: data.duration_seconds,  
  focused: data.focused_seconds,  
  score: data.focus_score,  
  userName: userName,  
};
```

For example, some field names that were pasted in JSON were different such as duration being stored as "duration_seconds" and also not all sessions included a timestamp. To handle this, I had to map the fields manually in the frontend before making the API request. I also

```
await addSession(  
  session.date,  
  session.duration,  
  session.focused,  
  session.score,  
  session.userName  
);
```

added a fallback to generate a date if one wasn't already there so that all sessions could still be displayed correctly.

This step became very important in my system because without it the data wouldn't be consistent and also the application would break when trying to display or calculate statistics. I also included some validation here to make sure that the JSON input was in the correct format before sending it to the backend which helped prevent any errors and made the whole import process more reliable.

User Interface & Experience

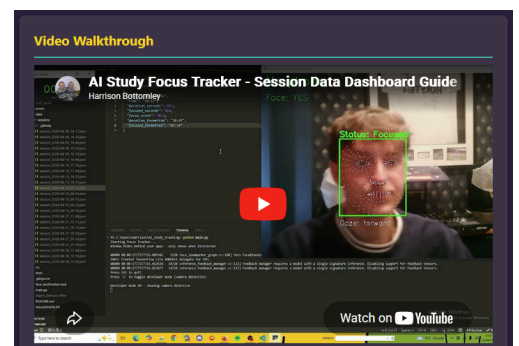
When designing my user interface i wanted it to feel simple and easy to use rather than overly complex and confusing. The main goal of this was to allow users to quickly understand their session data without needing to figure out how the system works.

I used a simple theme with bold colours to make the interface stand out a bit more which stuck throughout the whole project however i did keep on changing the colour theme frequently as i was undecided on which one fit the best. The theme and interface was partly based on design inspiration I had seen, but I kept the implementation simple so it didn't look over complicated as that is one thing personally i hate the most about some websites.

The navigation between pages is handled through a simple menu which allows the user to switch between viewing sessions, importing data, and reading the guide. The guide page was added to help explain how the AI tracker connects to the website, since this isn't something that would be obvious to a new user. I spent lots of time designing this page adding small features like shadows, professional boxes, a video walkthrough and copy buttons to make copying long links easier for the user.

I also made sure that the layout needed to work well on different sized screens by keeping elements flexible and by not relying on fixed widths too much.

Copy



Challenges and Solutions

Multi page structure

One of the main challenges that i faced when completing this project was trying to structure it as a multi page website. At the start i thought nothing of this and pictured it being fairly simple as it usually is however i realised i wasnt sure how to organise HTML pages with separate javascript files without everything becoming too confusing. I did initially have issues where scripts weren't loading as they should and pages weren't behaving as they should either. To fix this i separated each page with its own file such as a standard

`JS main.js` for the main sessions page but also `JS main-import.js` for the import page, `JS main-guide.js` for the guide page and `JS main-about.js` to handle the about section . This made each page independent and much easier to manage in the end.

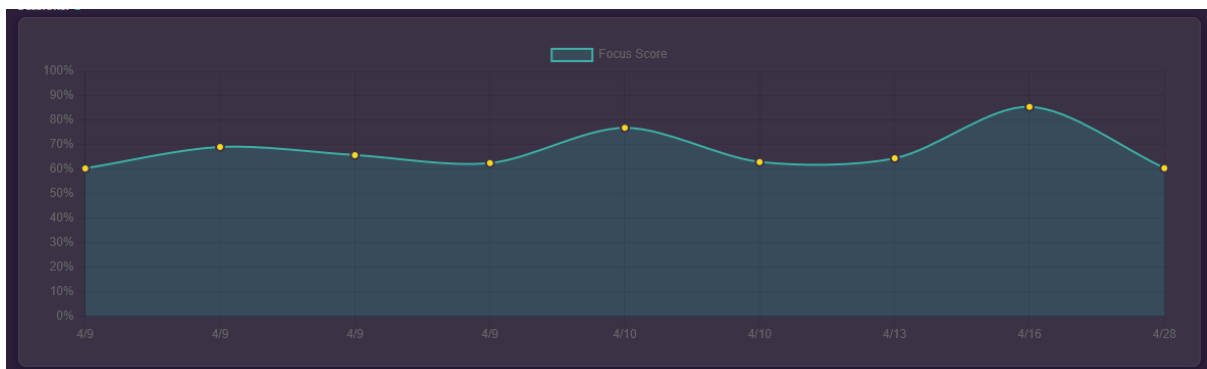
Frontend and Backend communication

Another issue I ran into was getting the frontend and backend to actually talk to each other. At one point, my requests just weren't working and nothing was showing on the page even though the server was running. After checking the console, I saw that the browser was blocking the requests. I found out this was because the frontend and backend were running on different local ports, so the browser treated them as separate and didn't allow the connection by default. To fix this, I added a small change to the backend to allow requests from the frontend. Once I did that, the data started loading properly and everything connected as expected.

Getting [Chart.js](#) Working

Another challenge I faced was adding the chart to visualise the session data. I needed to figure out how to take my session data and format it in a way the chart could use. The main issue was making sure the data was in the correct order and format, especially for the dates and focus scores. I solved this by sorting the sessions and then mapping them into labels and values that Chart.js expects. Once this was set up, the chart worked properly and updated whenever new sessions were added.

These challenges helped me better understand how different parts of a full stack application connect together and how small issues can affect the whole system. Throughout the project I worked in an Agile way, building and testing in small stages rather than trying to get everything done at once, which helped me catch problems early. For example, I tested API endpoints separately before connecting them to the frontend, and checked that session data was being stored correctly in the database before moving on to displaying it. This helped me catch issues early, especially with data formatting and frontend/backend communication



([chart.js](#) showing session data)

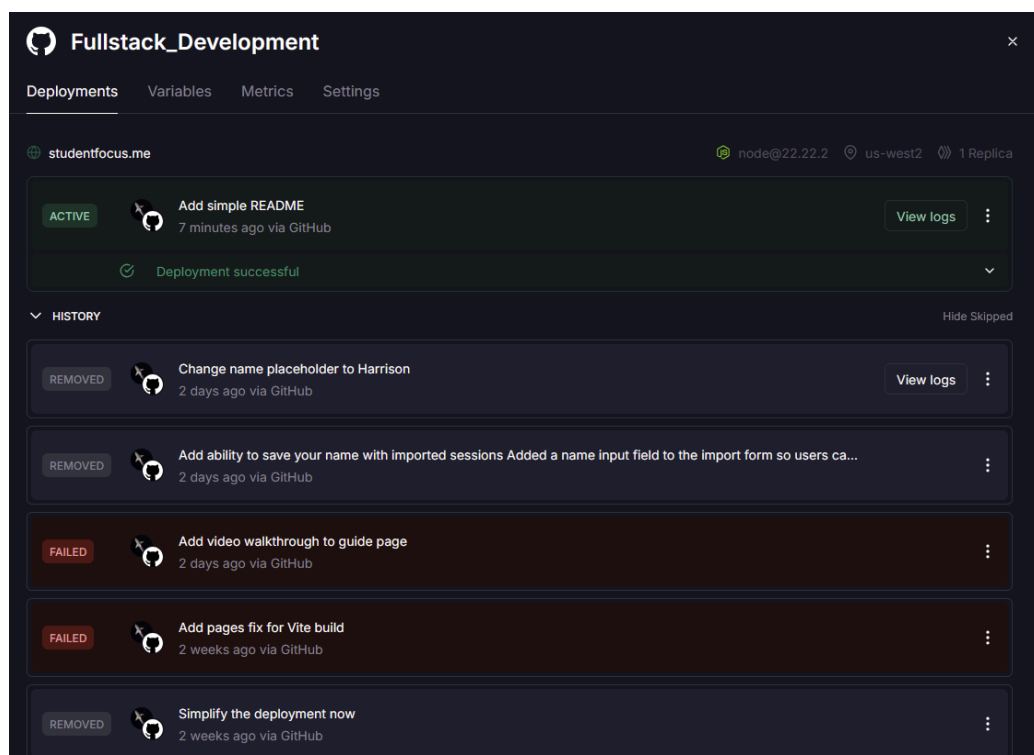
Deployment

The deployment part of my project turned out to be a lot longer than i thought, being used to deploying front end websites so easily i only thought the same about my fullstack project. I started with AWS (Amazon Web Services) setting up an EC2 instance and working through the configuration using Claude to help guide me through the extremely large AWS documentation. After a significant amount of time trying to get my project deployed and running, the site did eventually deploy perfectly just with the standard IP address URL it had given me. This was good however i wanted a better URL as the one it had given me looked very unprofessional and was not something i wanted to point anyone too

I realised i had my Github education benefits which included a free domain name through Namecheap, so I claimed the domain "studentfocus.me" and tried to point it at the AWS instance. This is where things got messy and confusing with AWS. Getting a custom domain to work with EC2 involves configuring DNS records, setting up an Elastic IP, and a few other steps that are easy to get wrong if you have not done it before. After spending a lot of time on it without getting a result, I made the decision to step back and look for a simpler deployment option rather than keep fighting with a setup that was not working.

I switched to a deployment site called Railway, which ended up being a much better fit for a project at this scale. The setup was straightforward and one of the features I found genuinely useful was that Railway deploys automatically whenever a commit is pushed to the GitHub repository. That meant I did not have to manually redeploy every time I made a change, which saved a lot of time during the later stages of development when I was still making small fixes.

Connecting my domain name studentfocus.me to Railway was also much simpler than the AWS attempt had been. Railway provides a CNAME target and Namecheap's DNS settings are clear enough that pointing the domain over only took a few minutes. The site has been running live at studentfocus.me since then without any issues.



Reflection

Overall i think the project achieved what i originally set out to do. The main goal of this was to improve how the session data could be understood and used which i think this project does very well. Before building this, the data was just numbers in a file which didn't really give much insight on its own. By creating the fullstack dashboard, i was able to turn that data into something more meaningful through statistics and visualisation, which makes it much easier to review study sessions over time.

One thing that went well was the overall structure of the project. Separating the frontend and backend early on made development much easier, especially when debugging. If something wasn't working, I could narrow down whether the issue was coming from the frontend display or the backend API instead of trying to figure it out in one large block of code. I also think the component based approach on the frontend worked well as components and ui_components files along with their own css and js files, this made it easier to manage things like session tiles, statistics, and charts without repeating code.

Another part that worked well was the data flow through the system. Once the import feature was working properly, the process became very smooth. The user can paste their JSON data, it gets processed and stored, and then immediately displayed with updated statistics and charts. This made the application feel responsive and complete rather than just a basic demo.

However, there are definitely areas that could be improved. One of the main limitations is that the system relies on manual JSON input. While this works for the project, in a more complete version it would be better to directly connect the AI tracker to the web application so that session data is sent automatically. This would remove the extra step for the user and make the system feel a lot more integrated in my opinion.

Another area for improvement would be expanding the analytics. At the moment, the system provides basic statistics such as average focus score, total time, and a simple chart. While this is useful, it could definitely be extended further with more detailed insights such as trends over longer periods, comparisons between sessions, or just way more advanced visualisations. I chose not to overcomplicate this part to keep the project manageable and aligned with the module requirements so that I wasn't trying to over engineer anything too much with the limited time i had.

Looking back, I also think I could have spent more time refining the user interface rather than adding extra features. While the design works and is easy to use, some parts of it could be improved visually to feel more polished. I changed the design multiple times during development, which shows that I didn't fully settle on a clear design direction early on.

Overall this project gave me a much better understanding of how a full stack application actually works as a whole system. Writing the backend on my own without any tools forced me to actually understand Express and REST APIs properly rather than just getting something working. The challenges I ran into along the way, whether that was CORS, deployment or the data format issues, all ended up teaching me something useful. Going

forward I feel much more confident building full stack applications, especially understanding how the frontend, backend and deployment all connect together as one complete system.